

Express.js Request Properties and Methods –

1. Introduction to Client–Server Requests in Express.js

In this tutorial, the focus is on **Express.js request (req) properties and methods**.

Focus on:

- **How clients send requests to the server**
- How Express.js reads and processes those requests using **request properties and methods**

Clients may send requests to:

- Read data
- Add new data
- Update existing data
- Delete data

To handle these effectively, we must understand **request properties (req)** and **request methods**.

2. Important Request Properties in Express.js

The most commonly used request properties are:

1. `req.params`
2. `req.query`
3. `req.body`
4. `req.cookies`

2.1 `req.params` and `req.query`

- Used for **route parameters** and **query parameters**
 - Data is embedded directly into the URL
-

3. `req.body` – Handling JSON and Form Data

Purpose

`req.body` is used to receive:

- JSON data
- Form data (URL-encoded)

This is extremely common in:

- APIs
- Web applications
- Mobile app backends

4. Enabling JSON Data in Express.js

To accept JSON requests, a middleware must be added:

```
app.use(express.json());
```

Why this is required

- Without this middleware, Express cannot read JSON request bodies
- It enables the entire app to accept JSON payloads

5. Sending JSON Data Using POST Requests

Route Setup

```
app.post("/about", (req, res) => {  
  res.send(req.body);  
});
```

- **POST** is used to **send data to the server**
- The server reads data using `req.body`
- The same data is sent back in the response for testing

6. Testing JSON Requests with Postman

Steps in Postman:

1. Select **POST** method
2. Enter URL: `http://localhost:3000/about`
3. Go to **Body** → **Raw**
4. Select **JSON**
5. Send data:

```
{  
  "name": "Yahoo Baba",  
  "age": 25  
}
```

6. Click **Send**

Result

- Server receives JSON
 - `req.body` captures it
 - Response returns the same JSON
-

7. Enabling Form (URL-Encoded) Data

To accept form data, another middleware is required:

```
app.use(express.urlencoded({ extended: false }));
```

Purpose

- Allows Express to accept:
 - HTML form data
 - `x-www-form-urlencoded` data
-

8. Sending Form Data Using Postman

Steps:

1. Method: **POST**
2. URL: `http://localhost:3000/about`
3. Body → **x-www-form-urlencoded**
4. Add key-value pairs:
 - `name` → Akshay Kumar
 - `age` → 25
 - `city` → Delhi
5. Click **Send**

Result

- Form data appears in `req.body`
 - Server returns it as JSON
-

9. Other Important Request Properties

9.1 `req.hostname`

- Returns the hostname of the request

- Example output:
 - `localhost`
 - `:::1` (local IP)
 - On a live server, it returns the actual domain IP
-

9.2 `req.ip` and `req.ips`

- `req.ip`: Single client IP
 - `req.ips`: Used when:
 - Load balancing
 - Multiple proxy servers
 - Mostly useful in production environments
-

9.3 `req.method`

- Returns HTTP method used:
 - GET
 - POST
 - PUT
 - DELETE

Example output:

```
GET
```

9.4 `req.originalUrl`

- Returns the full URL after hostname
- Includes:
 - Route
 - Query parameters

Example:

```
/about?name=abc
```

9.5 `req.path`

- Returns only the route path
- Excludes query parameters

Example:

```
/about
```

9.6 `req.protocol` and `req.secure`

- `req.protocol`: `http` or `https`
- `req.secure`:
 - `true` → HTTPS
 - `false` → HTTP

Used to detect secure connections

9.7 `req.route`

- Provides internal route information
- Includes:
 - Route path
 - Supported methods
 - Stack information

Useful for debugging and advanced routing logic

10. Request Methods in Express.js

10.1 `req.accepts()` – Content Negotiation

Checks which content type the client supports:

- HTML
- JSON
- XML

Example Usage

```
if (req.accepts("html")) {  
  res.send("<h1>Hello HTML</h1>");  
} else if (req.accepts("json")) {  
  res.json({ message: "Hello JSON" });  
} else if (req.accepts("xml")) {
```

```
res.send("<message>Hello XML</message>");  
} else {  
  res.status(400).send("Content type not supported");  
}
```

Use Case

- API decides response format based on client capability

11. Reading Request Headers

11.1 Using `req.headers`

Returns all headers:

```
res.send(req.headers);
```

Example data:

- Host
- Connection status
- Cache control
- Accepted formats

11.2 Using `req.get()`

Retrieve a specific header value:

```
req.get("host");  
req.get("connection");  
req.get("accept");
```

12. Validating Content Type Using Headers

Example Logic

```
if (req.get("content-type") === "application/json") {  
  res.send("Valid JSON data");  
} else {  
  res.status(400).send("Unsupported content type");  
}
```

Practical Use

- Validate request format
 - Prevent unsupported data from being processed
-

13. HTTP Method Importance (GET vs POST)

- **GET**
 - Used for reading data
 - No request body
- **POST**
 - Used for sending data
 - Required for JSON/form data

Incorrect method usage leads to:

- Unsupported content type errors
-

14. Summary

In this tutorial, we learned:

- How clients send requests to Express.js
- How to read:
 - JSON data
 - Form data
- Core request properties:
 - `req.body`
 - `req.hostname`
 - `req.method`
 - `req.originalUrl`
 - `req.path`
 - `req.protocol`
- Request methods:
 - `req.accepts()`
 - `req.headers`
 - `req.get()`
- Testing APIs using **Postman**
- Validating content types and request formats

This forms the foundation for:

- API development
 - Database integration
 - Secure request handling
-